

# The Two-Axis AI Engineering Maturity Framework

*A Practitioner's Guide to Assessing and Advancing Human–AI Collaboration in Software Engineering*

---

April 2026 —v1.0

*Includes implementation reference: Arpinine-Harness v1.4.0 by Arpinine*

## Abstract

---

Consider two teams using the identical model during the same week.

Team A relies on a senior engineer who describes a feature in a chat window, receives code, and spends two hours debugging the result.

In contrast, Team B's engineer utilizes a session governed by 52 active rules, a comprehensive spec with defined acceptance criteria, and an architecture plan that prevents writes until boundaries are established.

Consequently, the AI's initial attempt succeeds 95% of the time, keeping correction overhead below 30 minutes. Although both teams claim to use AI for development, their methods are fundamentally different.

This paper explores a two-axis framework distinguishing AI Engineering Process maturity (EP) from Product AI-Nativeness (AIN).

These independent dimensions are often conflated, which we consider the most prevalent strategic error in modern AI engineering. We illustrate these governance structures through Arpinine-Harness, an Arpinine-developed AI Coding Assistant plugin that serves as a comprehensive working example of the framework.

The fundamental argument remains transformative AI productivity is driven not by the choice of model, but by the governance infrastructure, including contracts, persistent rules, and feedback loops—that ensures each session builds upon the previous one.

# 1. The Problem With “Using AI.”

---

Ask ten engineers whether their team uses AI for development, and most will say yes. Ask what that means, and the answers span an enormous range, from occasional tab completion to fully governed multi-application builds where the AI operates within a formal contract system.

Both answers are technically correct, yet they describe situations with almost nothing in common.

This gap matters for a simple reason: without a shared vocabulary, organizations cannot tell where they stand.

Procurement decisions default to what shows up on invoices, model subscriptions and tool licenses, rather than the governance infrastructure that determines whether any of it compounds. And ‘AI-native’ has become a label applied to two completely different things: a product with a chatbot widget and a product whose architecture was designed from the ground up to support AI agents.

*The costliest mistake in AI engineering strategy is treating process maturity and product architecture as interchangeable on a single axis. They are independent. Confusing them produces teams that are impressive on one dimension and invisible on the other.*

A useful frame here comes from Harness Engineering: **Agent = Model + Harness**.

The model is the intelligence. The harness is everything that makes that intelligence safe to ship the schemas, policy gates, sandboxes, traces, and evals.

Crucially, the model is a commodity you buy by the token.

The harness is capital you build, own, and accumulate. Two teams running identical models will produce different outcomes because their harnesses differ. This is why governance infrastructure compounds and model upgrades don’t.

That sentence signals a paradigm shift that most organizations have not yet fully absorbed. If the harness is capital — something you build, own, and accumulate — then a competitive advantage in AI-assisted engineering does not come from which model you use or how large your engineering team is.

It comes from the quality and maturity of the governance system you have built around the model. For most software engineering work, the code your team produces is increasingly a byproduct of that system, reproducible from the spec and rules, regenerable by the AI.

The system itself is the primary asset. This claim is strongest for product software in domains where the code’s content matters less than its correctness and the speed at which it can be produced or changed. It is weaker in regulated systems where the code itself is the auditable artifact, performance-

critical systems where implementation choices are inseparable from outcomes, and highly specialized domains where the code embodies irreducible expert knowledge.

In those contexts, the harness remains valuable, but as an enabler of better code, not as a replacement for it as the primary asset.

A useful diagnostic lens before examining either axis: most teams spend most of their AI engineering effort on surface-level problems: prompt quality, model selection, too many tools, weak memory. These are real problems, but they are not binding constraints.

Structural problems run deeper: no governed workflow, poor context handling, no eval loop, and inconsistent handoffs between sessions.

Deeper still are strategic problems: vague success criteria, wrong autonomy level, unclear failure boundaries. And at the root: no sharp objective for what the agent should actually do; no systems of thinking applied to the build.

The two-axis framework is a tool for diagnosing which layer is actually limiting you — and for identifying the specific investment that would move you forward rather than just making the current layer slightly better.

Most organizations optimize layer 1 while layers 3 and 4 are where the real failures live.

The dangerous part is that layer 1 improvements feel like progress, faster responses, cleaner outputs, and better tool selection. They produce better results from a system that may have no clear objective, no system of thinking, and no real problem worth solving underneath it.

The two-axis framework is calibrated to surface-layer 3 and 4 failures, not to layer 1.

## 2. The Two-Axis Framework

---

Two dimensions. Independent below the top. They converge at the highest levels of maturity, but most organizations will never get there by accident — only by deliberately investing in both.

### Axis 1 — AI Engineering Process Maturity (EP)

EP is about how human–AI collaboration is governed. Not what model you use — that is irrelevant here. The question is: what constrains the AI's behaviour? Where does human responsibility end and AI responsibility begin? And critically: does each session leave the system better than it found it, or does every project start from scratch?

The same model, running within a disciplined EP 4 governance system, will outperform itself at EP 2 by a non-marginal factor.

The infrastructure around the model does most of the work.

## Operating Modes — When to Apply EP Governance

The EP levels describe how rigorously a session is governed. But not every task belongs in the same governance posture. Before applying the levels, it is worth establishing which of the three operating modes the work is in, because the appropriate EP level varies by mode, and applying execution-mode governance to exploration-mode work is one of the most common and counterproductive mistakes in AI-assisted engineering.

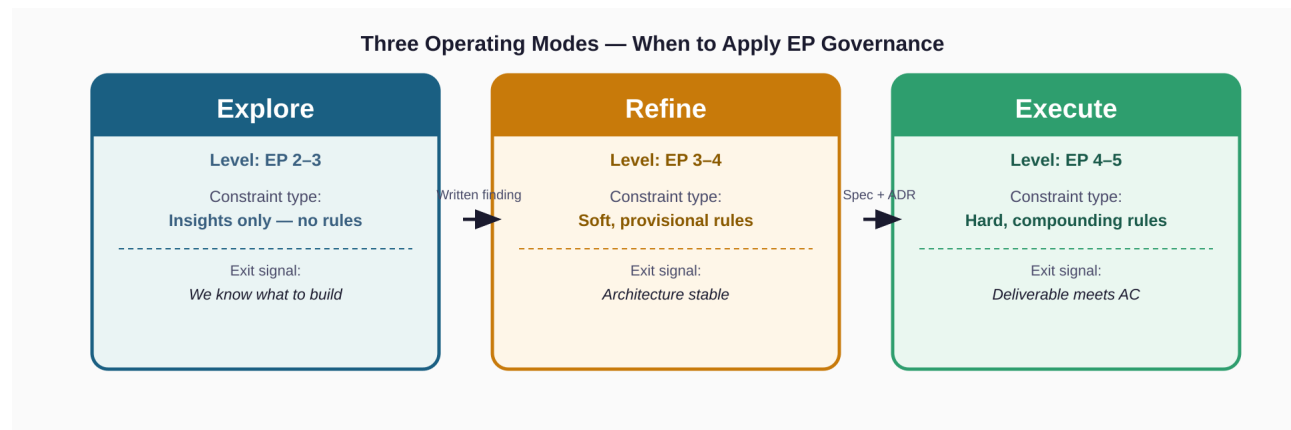
**Explore mode:** the goal is to discover what is possible. A spike to evaluate whether technology fits. A prototype to test a hypothesis. A creative generation task where the output space is undefined. Appropriate EP level: 2–3. Rich context, critical review, no compounding burden. Forcing measurable acceptance criteria onto open-ended investigation is not governance — it is obstruction.

The output of explore mode is a decision or a finding, not a deliverable. Explore mode does not compound through rules, but it should still produce durable insights. High-performing teams capture findings as lightweight patterns or decisions — not enforce constraints, but recorded judgments that inform the next mode of transition. The transition signal out of explore mode: we know what we are building and why.

**Refine mode:** requirements are partially known and evolving. A working system is being extended; a design is developing through feedback loops. Appropriate EP level: 3–4.

This is where most real engineering work happens, and it deserves more operational clarity than the other two modes.

Refine mode introduces provisional structure: completion criteria exist but are subject to revision; architectural decisions are made tentatively and documented as candidates rather than commitments; rules that accumulate here are scoped and soft — they constrain the current work without binding future decisions that the refine process has not yet settled. The progression across modes follows a constraint hardness scale: explore produces recorded insights with no enforced constraints; refine introduces soft constraints that reflect the current best understanding; execute applies hard constraints that persist and compound. The transition signal into execution mode: the architecture is stable, and the acceptance criteria are measurable and no longer changing from sprint to sprint.



The rule of lifecycle across modes follows the same constraint-hardness scale. In explore mode, there are no rules — only insights.

These are captured as lightweight findings (a paragraph in a shared log or a decision record without a formal rationale) but carry no enforcement weight. In refine mode, provisional rules are local in scope, explicitly marked as subject to revision, enforced within the current work but not propagated to other projects until they are promoted. The test for promotion is stability — if the rule has been held across three or more refined-mode sessions without being revised, it is a candidate for the persistent rule set.

In execute mode, rules are persistent, global, and compounding: they propagate across projects, fire on every session, and are never deleted, only superseded. The lifecycle prevents both rule fragmentation (duplicate rules at different stages) and premature hardening (explore-mode guesses locked into the persistent rule set before they have been tested).

**Execute mode:** implementation of a defined spec with measurable acceptance criteria. This is the mode that the rest of Section 3 describes in full. Full EP 4–5 discipline applies.

The governance overhead compounds return here precisely because the task is well-defined. Applying execution-mode governance too early — before the hypothesis is tested, or the architecture is stable — front-loads the overhead without the compounding return.

The mode of taxonomy does not replace the EP levels; it contextualizes them. EP 4 is not always better than EP 2.

Under-governance produces non-compounding systems.

Over-governance produces well-structured answers to the wrong questions.

The goal is not maximal governance, but correctly applied governance. The right diagnostic question is not “how do we get to EP 5?” but “what mode is this task in, and what EP level is therefore appropriate?” A team that runs at EP 4 in explore mode will produce a perfectly validated solution to the wrong problem. A team at EP 2 in execute mode will produce fast output that regresses every sprint.

## How to Identify Your Mode

Mode classification is the highest-leverage decision in the framework because the wrong mode invalidates the governance posture that follows. Yet it is the least specified part.

The following heuristics are not a formal algorithm—they are a quick diagnostic for teams that are genuinely uncertain about which mode applies.

You are in Explore mode if: success criteria cannot be stated in measurable terms; the architecture is unknown or genuinely open; multiple solutions are equally plausible, and the goal is to eliminate some of them; or the primary output is a decision rather than a deliverable.

The key signal is that you cannot fill in “Done when” with a specific, verifiable condition—not because you are being lazy, but because the answer is what you are trying to discover.

You are in Execute mode if: success is measurable and the criteria are stable; the architecture is settled and not under active revision; failure is expensive, and reversibility matters; and the output is a committed deliverable with a defined acceptance bar.

If you can write a spec with AC-NNN criteria that you would not revise during implementation, you are in Execute mode.

You are in Refine mode if: you can partially specify success, but the criteria are still changing; decisions are being made but are subject to revision; the architecture has a working shape, but its edges are still being defined; and the output serves as both a deliverable and a learning. This is the most common mode for ongoing product development. The risk in Refine mode is drift—staying here indefinitely because it permits both flexibility and apparent progress without the accountability of Execute mode.

A fast team-level check: ask each member independently to name the mode. If answers diverge, the mode is genuinely ambiguous, and the first task is alignment, not governance. Persistent divergence—where engineers say “Explore” and managers say “Execute”—is usually a sign of misaligned expectations about what the work is producing, which is a more urgent problem than deciding which governance posture to adopt.

## Axis 2 — Product AI-Nativeness (AIN)

AI Nativeness is often misunderstood, so it is worth being blunt: having AI features does not make a product AI-native. A chatbot, a recommendation engine, a content-generation block — these are AI features. They sit atop an architecture designed for human users.

AIN asks a different question: can an AI agent compose operations within this product, execute them, observe the results, and adjust based on those results? That requires architectural intent, not feature addition.

Most products marketed as AI-native in 2026 score low on AIN.

The label is being applied to products where AI is visible, not to products where AI has operational authority. This is not pedantry — the architectural gap between those two things is enormous and expensive to close after the fact.

## Why the Independence Matters

The two axes can move independently, and when they do, they produce specific failure patterns that most teams do not have a name for:

- **Pattern A — Capable product, chaotic process:** The team is shipping genuinely impressive AI-first products, but the engineering process behind them is chaotic. Errors recur. Building velocity is unpredictable. The product’s own AI sophistication has outpaced the team’s ability to govern its construction.
- **Pattern B — Disciplined process, wrong product:** The team has built real governance discipline — persistent rules, formal plans, compounding feedback. But the product they are building treats AI as a feature layer. All that governance infrastructure is pointed at the wrong architectural target.

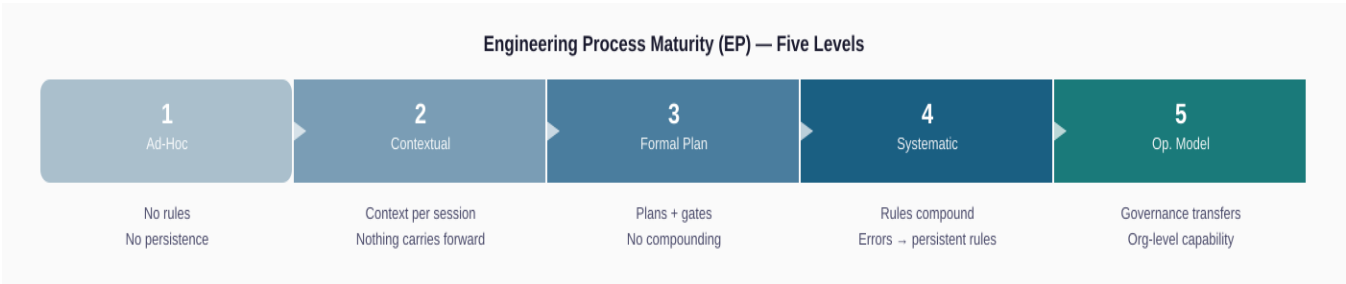
*The matrix below shows how the four quadrants map. Most organizations currently sit in the bottom half — the “Starting Point.” The goal is the top-right quadrant, the Converged Frontier, but reaching it requires deliberate and independent investment in both axes. Advancing along only one-dimension leaves significant value on the table.*



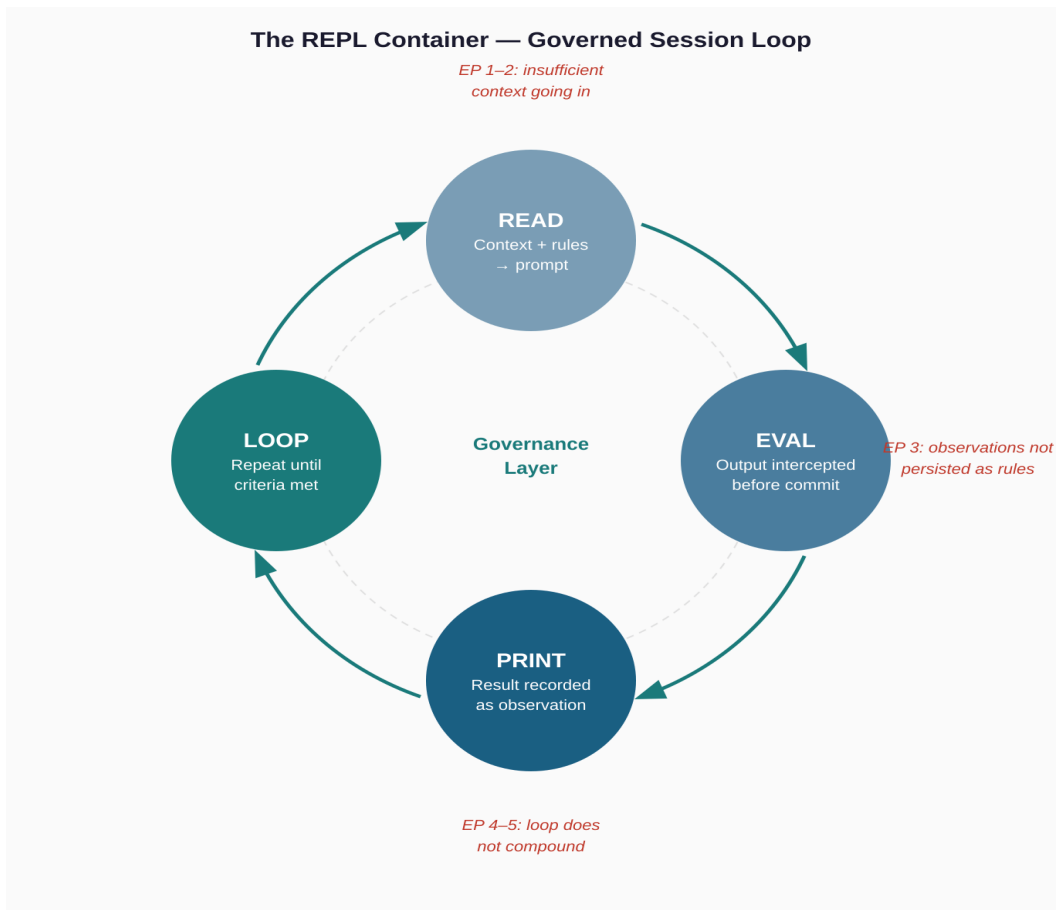
### 3. AI Engineering Process Maturity — Axis in Depth

Five levels. Each one requires a specific change in how you work, not which tools you buy.

Most transitions are harder than they look from the outside because they require discipline rather than capability. The AI can handle EP 4 work on day one. The question is whether you have built the structures that let it.



### The REPL Container: A Mechanical Model of a Governed Session



Before walking through the levels, it helps to have a concrete picture of what a well-governed AI session looks like mechanically.

The REPL Container model is a good frame: every session is a loop with four stages.

**Read** — you translate context into a prompt the AI can work with.

That means task intent, architectural constraints, active rules, and relevant code. What goes in here determines the ceiling of what comes out.

**Eval** — when the AI produces output, an intercept layer catches it before it touches anything. Constitution checks, architecture boundaries, and postcondition gates.

**Print** — the result, whether a success or a drift finding, gets recorded as a structured observation and fed back into context.

**Loop** — repeats until the task meets its stated completion criteria or hits a termination condition.

The EP levels essentially serve as a scale for how rigorously this loop is governed.

EP 1 and 2 are reading failures — not enough context going in.

EP 3 is a Print failure — results are not being captured as persistent rules.

EPs 4 and 5 are Loop failures — the cycle runs, but it does not persist across sessions. Knowing which stage is breaking down speeds up diagnosis compared to guessing.

Harness engineering literature breaks the loop into seven layers: instruction (what the AI is told and constrained to do), tools (its action vocabulary), memory and retrieval (what persists and what gets pulled in), execution (the sandboxed environment where tool calls actually run), policy and approval (the gate between planning and acting), observability (traces, costs, metrics), and evaluation (the quality signal that closes the outer loop).

The EP levels map directly onto this.

Practitioners of EP 1 and 2 are working with layers 1 and 2 informally.

EP 4 means you have explicitly engineered layers 5 and 6.

EP 5 means you are governing layer 7 as a managed system.

AIN maturity asks a different question: is the product you are building itself a well-designed harness — one that exposes its own layers 1 through 4 to external agents?

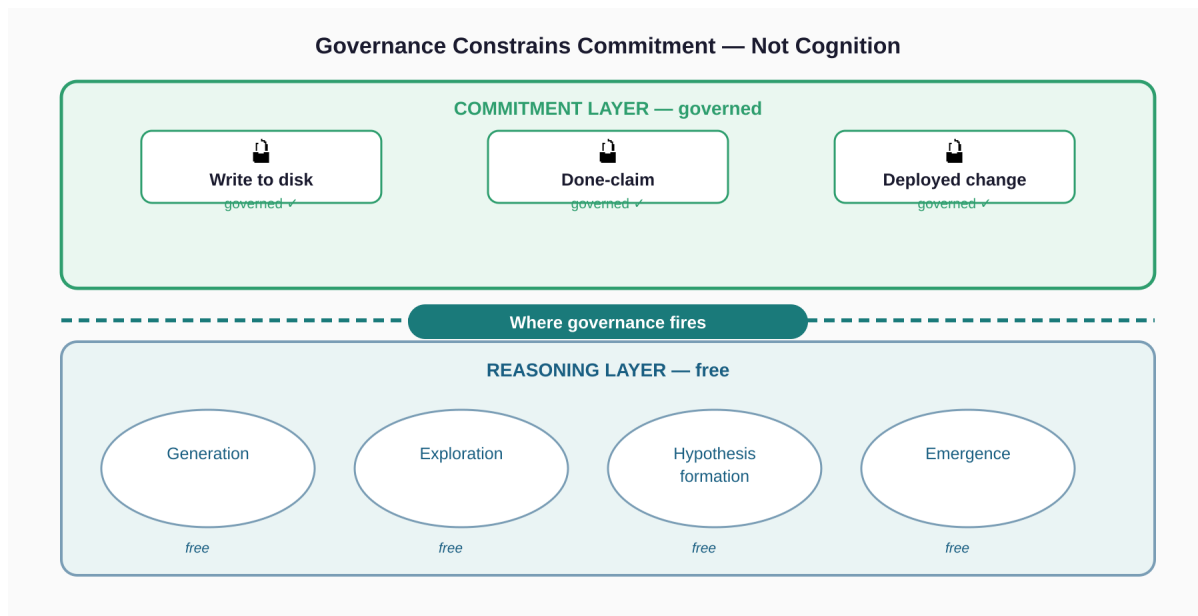
Governance should primarily constrain commitment, not cognition.

Constraints on cognition — tool access limits, retrieval boundaries, cost budgets, safety guardrails, should be minimal and deliberate, applied where the risk of unconstrained reasoning is demonstrably higher than the cost of reduced emergence. These are legitimate constraints, but they are different in kind from the governance infrastructure this framework describes.

The rules, contracts, and policy gates here fire when AI output is about to become an artifact — a write to disk, a done claim, a deployed change.

The stochastic, emergent, context-sensitive properties of LLMs are exactly what you want active at the reasoning layer.

The governance layer exists to ensure that when reasoning produces something worth committing, the commitment is auditable, attributable, and governed by accumulated rules.



Concretely: a rule that says “never hardcode credentials” does not reduce emergence.

A postcondition that says “the build must pass” does not make the AI less creative in solving the problem. A hook that blocks writing to implementation paths until architecture boundaries are defined does not constrain how the AI thinks about the architecture.

The governance layer is the banks of the river, not the water. The hooks-on-write model that Arpinine-Harness uses is architecturally correct for precisely this reason: it leaves reasoning free and governs commitment.

The practical implication: a governance system that constrains prompts, limits tool access during exploration, or enforces a spec format during a spike operates at the wrong layer.

The highest-leverage governance acts at the point of commitment; constraints on reasoning should be minimal, deliberate, and justified by a specific risk that outweighs the cost of reduced emergence. If your governance infrastructure feels like it is making the AI less capable, the first diagnostic is not “reduce governance” — it is “which layer is this constraint operating at, and is that the right layer for this task?” The answer is often that a commitment-layer constraint has been applied too early in the task lifecycle, before the mode has transitioned to execute.

## Mode Transition Gates

The boundary between operating modes is where the most expensive failures occur. Teams get stuck in exploring mode indefinitely — running spikes that never produce a decision and prototypes that drift into production without governance. Or they apply execution-mode discipline too early, constraining exploration before the hypothesis is tested and losing the emergence that made the investigation valuable.

Two lightweight transition gates prevent both failure modes. The Explore → Refine gate requires a written finding: what was learned, what was decided, and what was ruled out. This does not need to be a formal document — a paragraph in a shared log is sufficient. Its purpose is to force a deliberate decision about whether exploration has produced something worth building. Without it, explore mode silently becomes execute mode, and the resulting system has no governing spec.

The Refine → Execute gate requires two things: a spec with at least one measurable acceptance criterion and at least one architectural decision documented with rationale.

These are the minimum artifacts that signal the work has crossed a threshold into well-defined territory.

In Arpinine-Harness terms: /spec-new + /spec-plan must be run and substantively completed before EP 4 governance is applied. Running them with placeholder content passes the structural check but defeats the purpose.

The transition gates also clarify why the REPL Container loop described above sometimes feels like overhead: it is overhead in explore mode.

The Read stage assumes a well-specified context.

The Eval stage assumes postconditions are worth checking.

The Print stage assumes observations are worth persisting. In explore mode, none of these assumptions hold, and trying to enforce them produces friction without governance value.

The loop becomes useful at the Refine → Execute transition, when there is something specific enough to govern.

## Wrong Mode Diagnostics

Mode misclassification is a failure mode, and it is worth giving it observable signals rather than leaving teams to discover it through accumulated pain.

Three patterns reliably indicate that the mode label does not match the work:

Governance feels like friction. If the spec, plan, and gate structure feel like obstacles rather than scaffolding, the most likely explanation is not that the team lacks discipline — it is that the work is in explore or refine mode and execution-mode governance has been applied prematurely.

Governance is correct for a different task. The recovery is not to abandon governance but to step back to the appropriate mode and its lighter posture, then re-enter execute mode when the preconditions are genuinely met.

Errors repeat across sessions. If the same class of failure recurs despite correction, the work is in execute mode without execute-mode governance.

Errors should generate persistent rules; if they do not, the system is not compounding.

The recovery is explicit attribution and rule creation — running /spec-retro and extracting at least one rule from the recurring pattern.

Output is precise but irrelevant.

If the delivered work technically meets its stated criteria but does not move anything that matters, the explore mode was treated as execute mode too early. The acceptance criteria were formalized before the hypothesis was tested. The recovery is to treat the deliverable as a finding rather than a result: document what was learned, revise the hypothesis, and re-enter from explore or refine mode with updated intent.

Refine mode persists without progress signals. If architectural decisions are being made and revised repeatedly without converging, and completion criteria keep changing, the team may be in the refine-mode-forever trap — using the flexibility of refine mode to avoid the accountability of execute mode. The diagnostic question: has any acceptance criterion remained stable for two full sprints? If not, the work has not yet earned execute-mode governance, but the team should be asking why convergence is not happening.

## **EP Level 1 — Ad-Hoc Interaction**

At Level 1, every session starts from zero. No context carries over. No rules constrain what the AI can do.

The interaction is reactive — ask something, get something, maybe use it.

The ceiling is low, not because the AI lacks capability, but because the human gives it almost nothing to work with. Without architectural context, active constraints, or completion criteria, the AI produces output that fits its training distribution.

That output may be technically correct in the abstract, but it is wrong for the specific codebase, pattern, or constraint in front of it.

The honest signal that you are at Level 1: the same question gets asked again next month. Nothing from the first session made the second session cheaper.

## **EP Level 2 — Contextual Direction**

Level 2 practitioners have figured out that context matters. They provide architecture descriptions, API schemas, existing code, and relevant constraints — and the quality of AI output jumps noticeably. This is a real improvement. It is also where many practitioners stop because the sessions feel productive.

The problem is that nothing carries forward. Each session is self-contained. The same errors fixed last week recur this week because there is no mechanism to prevent them.

The test is simple: how often do you see the same class of mistake across different sessions? If the answer is frequently, you are at Level 2, regardless of how good the individual sessions feel.

A data platform team provides a detailed Kafka schema and quality rules before generating a pipeline. Good output. Two weeks later, a new pipeline task makes the same null-handling error that was caught the first time — because the lesson lived in someone’s memory, not in the system.

## EP Level 3 — Formal Planning

At Level 3, something structurally important changes: you write the completion criteria before the AI starts. Done when the build passes. Done when these three endpoints return the expected schema. Done when the integration test covers this boundary. Architectural decisions are documented with rationale, not just made.

This shift matters more than it sounds. At Level 2, you evaluate AI output by feel — does this look right? At Level 3, you evaluate it against a stated criterion. That makes the AI’s claim of ‘done’ falsifiable rather than merely assessable. It is a real change in the nature of the governance relationship.

What Level 3 still lacks: the lessons do not stick. Errors get corrected; criteria get tightened, but none of it carries over to the next project. You are running a well-governed session, not a compounding system.

### ► Arpinine-Harness implementation (*Arpinine — agent-align v1.4.0*)

**/spec-new + /spec-review:** Creates spec.md from feature request, then reviews it for measurability and AC format before engineering starts. Each acceptance criterion must match the AC-NNN: pattern — enforcing structured postconditions rather than free-text ‘done’ claims.

**/spec-plan:** Produces plan.md with module boundaries, dependency rules, and testability sections as required gates before implementation. The architecture-readiness hook blocks write to implementation paths until all three sections are substantively present — making it structurally impossible to skip formal planning.

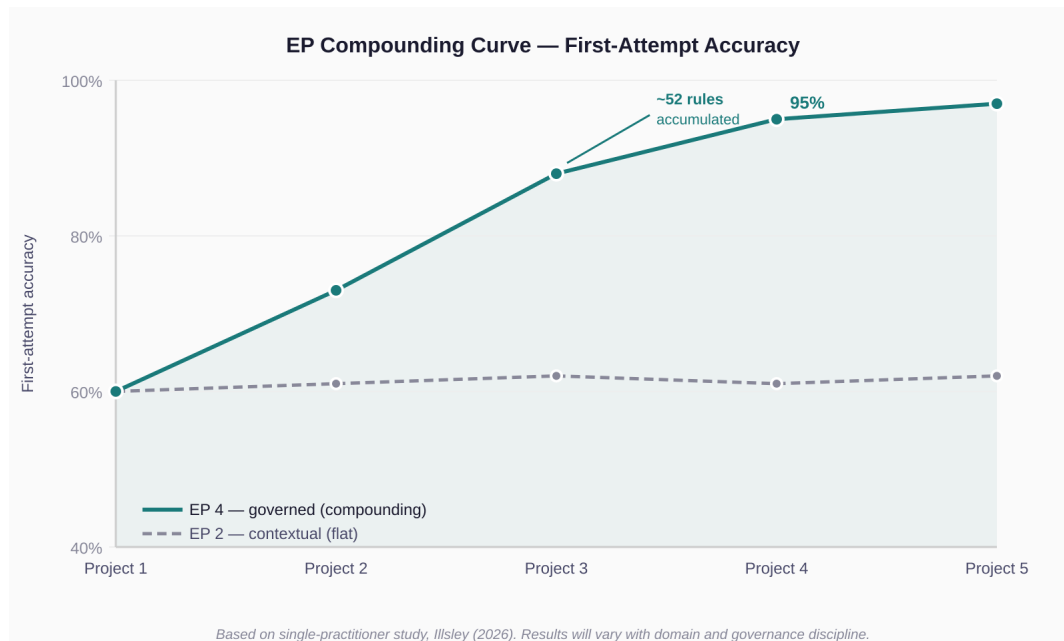
**check-constitution.sh:** PreToolUse hook that blocks write to spec.md containing implementation language (30+ keywords). Enforces the boundary between intent and execution at the artifact level.

## EP Level 4 — Systematic Governance (The Compounding Threshold)

This is the transition that changes economics. At Level 4, when the AI makes an error, you do not just fix it. You ask a specific question: was this a precondition failure—did the AI lack the context it needed? If so, the responsibility is yours. Or was it a postcondition failure—did the AI deviate from explicit requirements, which means a rule needs to be written? The answer differs in each case and leads to a different corrective action.

The correction is then written as a persistent, machine-readable rule. Not a comment in a doc somewhere. A rule that fires on every subsequent session, across every future project. The rule set accumulates. Each new project starts at the accuracy ceiling set by everything learned before it.

That is what compounding means in this context—not that the AI gets smarter, but that the governance system does.



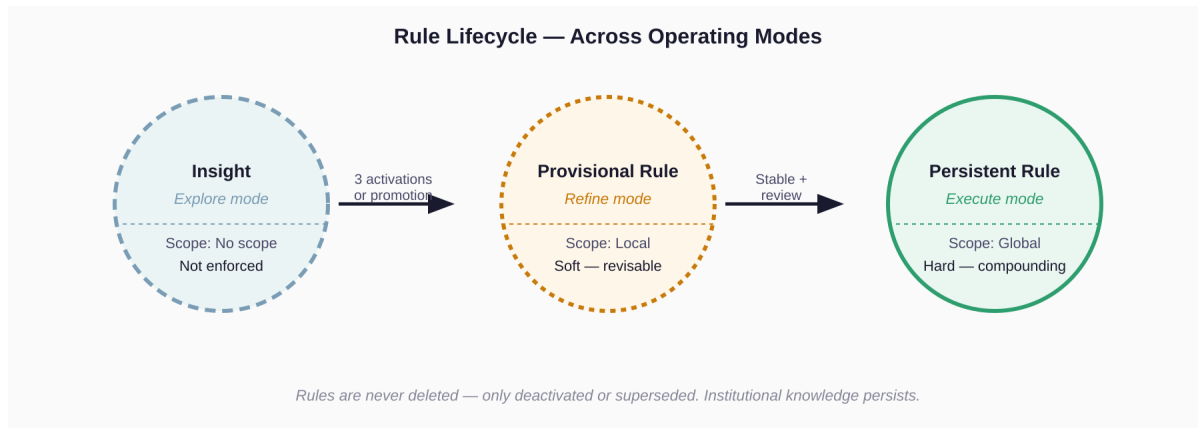
One risk that accumulation creates is worth naming explicitly here, because the framework celebrates rule growth without fully acknowledging its cost curve.

A set of 50 well-maintained rules outperforms a set of 200 that includes contradictions, stale constraints, and redundant variations of the same principle.

The compounding property depends on rule quality and trust, not just quantity. When practitioners encounter violations, they cannot trace a clear rationale; they begin treating the governance system as advisory rather than authoritative — and once that happens, the compounding property collapses.

Rule accumulation must therefore be paired with rule governance: the capacity to detect contradictions, identify abstraction opportunities, retire stale constraints, and maintain the legibility of the rule set as it grows.

This is the governance-of-governance problem, and it is addressed operationally in Section 5 and via agent-assisted rule management in the Arpinine-Harness implementation.



► **Arpinine-Harness implementation** (*Arpinine — rule governance, roadmap*)

**/spec-rule-health:** Runs the full rule set governance audit. Invokes the agent against the active rule set to produce: a contradiction report (pairs of rules with semantic tension), an abstraction candidates list (rule clusters addressing the same principle), a staleness report (rules below an activation threshold), and a redundancy report (near-duplicate rules). Output is a structured report requiring human decision before any changes are made.

**Contradiction pre-commit gate:** When add-rule is called, the agent checks the new rule against the active set for logical conflict before writing the rule file. Conflicts block the write and surface both rules with the specific tension between them. The human must resolve the conflict or create an ADR documenting the known tension before proceeding.

**Provisional rule state:** Rules begin as provisional — local-scoped, time-bounded, not propagated across projects. Promotion to persistent requires three activations across different sessions or explicit human promotion. This prevents explore-mode guesses from entering the compounding rule set before they are tested.

**/spec-rule-health abstract:** Takes a cluster of rule IDs, generates a proposed general rule that subsumes them, presents for human review, and if approved: writes the new general rule, supersedes the cluster members with the new rule as the superseding reference, and updates the ADR index. Institutional knowledge is preserved in the superseded rules; the active constraint is the cleaner general principle.

**Rule activation telemetry:** Each rule file gains metadata: last-activated, activation-count, projects-activated-in. Populated by drift-detector on every run. Fed into /spec-rule-health for staleness analysis. Rules below the activation threshold over a configurable window are flagged for review—not deleted but surfaced for human decision about whether they remain relevant.

*The Level 4 threshold is not about capability — it is about discipline. Every ‘done’ claim is verified against the stated postcondition. Every error generates a root cause analysis. Every lesson generates the rules of a candidate. Every session leaves the system smarter.*

The policy gate is what makes the discipline stick without relying on memory or vigilance.

In harness engineering, every tool call passes through a policy layer between the AI’s planning output and execution: permission checks, input validation, dry-run defaults for destructive operations, and audit logging.

The same pattern applies here. EP 4 practitioners wire hooks to run before every edit — constitution checks, architecture readiness, and active rule validation. Violating the policy becomes harder than the compliant path, not because someone reviews every output, but because the structure prevents it.

The economic effect is real and non-linear.

Early projects with an immature rule set might see 60% first-attempt accuracy and hours of correction overhead per feature. Later projects, operating with 50+ accumulated rules, routinely run at 95% first-attempt accuracy, with correction overhead measured in minutes. The model did not improve. The governance infrastructure was important. That is what non-linear returns look like in practice.

► **Arpinine-Harness implementation** (*Arpinine — agent-align v1.4.0*)

**/spec-audit (drift attribution):** Classifies every finding as PRECONDITION FAILURE (spec or plan was unclear; refine the artifact), POSTCONDITION FAILURE (spec was explicit; implementation deviated; candidate for a new rule), or RULE VIOLATION (matches an active rule; fix the code). This three-way classification is the operational form of the attribution discipline prescribed by the framework.

**rule-manager (add-rule):** When /spec-audit identifies a postcondition failure, it flags it as a rule candidate. /spec-retro extracts lessons from completed work, classifies each as a rule candidate, and persists confirmed rules to .specify/rules/ via rule-manager. Rules are never deleted — only deactivated or superseded — ensuring the compounding property is preserved across projects.

**drift-detector (rule check):** Before creating a new ADR for a drift finding drift-detector runs rule-manager. Check-rules. If the pattern is already captured by a rule, it becomes a RULE VIOLATION requiring code correction—not a new ADR. Known patterns do not regenerate governance overhead.

## EP Level 5 — Operating Model Design

Level 5 treats the governance framework itself as a product.

You make deliberate decisions about when to sprint through implementation and when to slow down for testing, when it makes sense to temporarily break a rule with a documented recovery plan, and how to prevent the whole system from walking out the door with its creator.

The test that separates Level 5 from a talented Level 4 individual is simple: could someone else walk in tomorrow and run at Level 4 quality from day one, using only what is written down? If the answer is no — if the institutional knowledge lives in someone's head rather than in machine-readable artifacts — then the organization is at Level 2, even if the individual is not.

The individual did not change. The governance infrastructure simply wasn't there.

► **Arpinine-Harness implementation** (*Arpinine — agent-align v1.4.0*)

**/spec-status --onboard:** Generates a new team member brief from the current governance artifacts: active rules, open ADRs, eval thresholds, architecture boundaries, and drift status. The brief is derived from machine-readable artifacts, not hand-written documentation — meaning it reflects the actual current state of governance, not the state as of the last time someone updated a wiki.

**Governance table:** /spec-status produces a project-wide governance table covering acceptance criteria status, drift findings, ADR coverage, eval results, and architecture readiness. This is the operational equivalent of the ‘build economics tracked at process level’ indicator for EP 5 — making the governance system’s health visible rather than inferred.

**Artifact schemas:** Machine-readable YAML schemas (spec-schema.yaml, adr-schema.yaml) enforce substance, not just structure. ADRs must include ## Context, ## Decision, ## Consequences. ACs must match the AC-NNN format. This distinguishes substantive governance from compliance theatre — artifacts that pass schema validation are artifacts that would actually inform a new practitioner.

## 4. Product AI-Nativeness — Axis in Depth

---

AI Nativeness maturity is about architecture, not features.

A product scores high on AIN when an AI agent can walk in, compose a sequence of operations, run them, see the results, and refine its approach based on what happened — without a human mediating every step. That requires deliberate architectural decisions. It cannot be bolted on.

### AIN Level 1 — AI-Free or Invisible

No AI in the product. Worth saying clearly: this is not a criticism. Plenty of excellent products are at Level 1 and should stay there.

The point is that moving beyond Level 1 requires a deliberate design decision — you cannot get there incrementally by adding features.

### AIN Level 2 — AI as Feature Layer

The product includes AI features: a chat interface, a content generator, and a recommendation module. These add real value. Architecturally, though, they sit atop a system designed for human users. The data model, event flow, and operational logic — none of it was designed with AI agents as first-class operators.

This is where most ‘AI-native’ products actually sit. The AI is visible to end users, which is what drives the label. But AI agents do not have architectural authority within the system. They can respond; they cannot act and observe simultaneously.

## AIN Level 3 — AI-Callable Architecture

Level 3 is a real threshold.

The product now exposes a structured vocabulary that AI agents can use directly — APIs, tool schemas, and domain-specific interfaces. Operations are composable. The system accepts AI-initiated commands, not just user inputs. This does not happen by accident; it requires deliberate design investment and usually forces a rethink of existing service boundaries.

The remaining gap at Level 3: the agents can act, but they cannot see what happened. There is no feedback channel from the product to the AI. The agent fires a command into a system that does not tell it what changed.

► **Arpinine-Harness implementation** (*Arpinine — agent-align v1.4.0*)

**AIN Assessment block in spec-schema.yaml:** Specs at  $AIN \geq 3$  must populate `agent_callable_operations` with name, input schema, output shape, and failure modes for each operation. This ensures that the AI-callable vocabulary is explicitly designed before implementation begins—not discovered after the fact.

**Human-in-the-Loop Gates:** The AIN assessment block requires an explicit declaration of decisions that require human approval before an agent proceeds. This makes the human-agent boundary a governed contract rather than an implicit assumption.

**Evaluation Required gate:** Specs with  $AIN \geq 3$  and `Evaluation Required: YES` block implementation until `eval-plan.md` exists with metrics, thresholds, datasets, and a pass/fail policy. AIN Level 3 without an evaluation contract is not a governed system — it is an ungoverned one that happens to have an API.

## AIN Level 4 — Composable with Feedback

This is the true minimum threshold for AI-native architecture.

The agent can compose operations, execute them, and return real information about what happened — not a status code, but a structured observation: which checks passed, which raised alerts, and what state changed. It uses that to refine the next iteration. The loop closes.

Consider an operations platform at a mining company. It exposes a DSL for composing inspection workflows. An AI agent composes a sequence of sensor queries and anomaly checks, runs it, receives a structured event stream, and adjusts its next iteration based on what triggered it. No humans in that loop. That is Level 4. Everything below it is an agent that fires commands into the dark.

One property of Level 4 systems that is consistently underbuilt: loop safety. In production, the most common agent failure is not a wrong answer — it is a loop. The same tool is called repeatedly with the same arguments. Two operations alternate indefinitely. An agent re-reading its own output without making progress. These failures produce partially applied state changes, which are among the hardest production bugs to diagnose and reverse.

Three layers of defence are needed, and all three matter:

Hard limits. Every agent run gets a step budget, a token budget, and a wall-clock budget. Exceed any of them, and the run escalates with a structured trace — never completes silently. These are the floor, not the solution, by themselves.

Behavioural detection. The system watches the trace for duplicate or near-duplicate tool calls and injects a reflection prompt when the pattern fires, rather than letting the agent burn through its remaining budget first.

Structural prevention. Write tools get idempotency keys so retries are no-ops. State-changing operations must advance the observable state or trigger re-observation. The system forces a different approach after N failed attempts at the same one.

A composable agent interface without all three of these will eventually produce a production incident, usually at the worst possible time.

Getting the loop closed is the structural requirement.

Getting it production-safe requires four additional properties, collectively referred to as R.E.S.T: Reliability, Efficiency, Security, and Traceability:

**Reliability.** Retried operations must not corrupt the state. Idempotency is mandatory when agents run autonomously within a product.

**Efficiency.** Token consumption, API calls, and compute time per cycle must be budgeted and visible — not discovered after the fact on an infrastructure invoice.

**Security.** Agent operations must run under least-privilege constraints with proper sandboxing and I/O filtering. An agent with excess permissions inside a feedback loop carries a significant blast radius.

**Traceability.** Every decision needs an attribution record that is queryable at any point in the run's history. Debugging a loop failure without traces is essentially impossible.

**Structural** composability and feedback get you to AIN 4 on paper. These four properties get you there in production.

Beyond these four properties, real-world production experience across multiple agent harnesses reveals a critical fifth requirement: context continuity. Without deliberate management of the agent's

working set, even a well-designed AIN Level 4 feedback loop degrades under sustained load — not catastrophically, but gradually and expensively.

## Context Continuity — The Fifth Production Requirement

Comparative analysis of production agent harnesses — including Claude Code, Pi, OpenClaw, and Letta — reveals a fifth operational requirement that R.E.S.T. does not cover: context continuity. Every one of these systems independently converged on the same finding: an AIN Level 4 feedback loop that works at demo scale will degrade under production load if the agent runtime does not actively manage its own working set. As sessions grow, file reads expand, tool outputs accumulate, and sub-agent calls multiply. Without active management, the agent loses coherence — not catastrophically, but gradually, in ways that are hard to detect and expensive to diagnose.

The convergent solutions across all four harnesses point to four specific requirements for context continuity in a production AIN Level 4 system. File reads must be capped and paginated — hard limits on read size with explicit continuation mechanisms, not unbounded reads that flood the context window with boilerplate. Oversized tool results must be offloaded outside the context window and replaced with compressed previews the agent can expand on demand; a 60KB grep result that consumes a third of the context budget on the first turn of a session makes every subsequent turn worse. Session compaction must be triggered by token pressure and must respect tool-call boundary integrity — compaction that orphaned a tool call from its result corrupts the agent’s understanding of what has already happened. Sub-agent sessions must be isolated from parent transcripts unless forking is deliberate, preventing cascading context of pollution across agent boundaries.

There is also a deduplication principal worth naming explicitly: known observations should not consume context budgets the same way new ones do. Claude Code implements this as read deduplication — if the agent re-reads the same file at the same range and the file has not changed, a stub is returned rather than the full content. Arpinine-Harness’s rule system applies the same logic at the governance layer: drift-detector checks rules before creating ADRs, so known failure patterns do not regenerate governance overhead. The underlying principle is identical — compounding systems must not reprocess what they already know.

What makes this finding compelling is not the mechanisms themselves but how they were discovered. These four engineering teams did not coordinate. They faced the same physical constraint — a finite context window under production load — and independently arrived at the same five solutions. This is convergent evolution in engineering: just as fluid dynamics produces fins on both sharks and dolphins, token pressure produces identical mechanisms in every harness that takes production seriously. It is not a prescribed best practice checklist. It is what happens when you build at scale.

The most important architectural nuance of convergence analysis surfaces is not that tool results are offloaded — it is when. Claude Code does not wait for context pressure. Before every API call, oversized tool results are written to disk and replaced with a 2KB preview in context. A 60KB grep

result is kicked out on the very first turn, before anything is close to full. Experience shows that tool output is the primary cause of the context window being closed, so the harness acts first. The implication for AIN Level 4 design: tool result offloading should not be a pressure-relief valve. It should be the default path, with in-context retention as the exception. Tool output is presumptuously disposable — useful for the current reasoning step, archivable thereafter.

Sub-agent session isolation follows the same logic. All four harnesses default to not pass parent session context to child agents — not because engineers forgot to wire up inheritance, but because parent session noise overwhelms child task objectives. Context is not inherited. Context must be designed. The deliberate absence of inheritance is itself a governance decision, as consequential as any rule in the ruleset. A child agent that receives only its task string, and the current rule set will outperform one that inherits a sprawling parent transcript every time.

*Context continuity is not a feature to add later. A product that achieves AIN Level 4 composability and feedback in short sessions but degrades after 30 minutes of production operation has not achieved AIN Level 4. It has achieved a demo. Fifty years of computing taught us that the best memory management is the kind the running program never has to think about. The same principle applies here: the goal is not to show the agent everything — it is to give it the right working set at the right time, managed by the harness, invisible to the agent’s reasoning layer.*

► **Arpinine-Harness implementation** (*Arpinine — agent-align v1.4.0*)

**Feedback Channels (AIN assessment block):** The spec template requires explicit declaration of how the system captures signals about agent quality — not as a free-text field, but as a structured section that the evaluation-governor references when determining whether evaluation contracts are sufficient.

**evaluation-governor:** Blocks implementation completion on failing eval thresholds and tracks eval regression. For AIN Level 4 systems, this means the feedback loop the product provides to its agents is itself under governance — regression in feedback quality is detected, not just regression in output quality.

**EP–AIN axis alignment:** A team building an AIN Level 4 product while operating at EP Level 2 is the most dangerous configuration in the framework. Arpinine-Harness’s /spec-status governance table makes this mismatch visible by surfacing EP-level indicators (rule density, ADR coverage, drift attribution counts) alongside AIN-level indicators (eval plan status, feedback channel definition) in a single view.

## AIN Level 5 — Self-Improving System (Frontier)

At Level 5, the feedback loop becomes self-reinforcing. Agent operations generate experience data that improves agents. The boundary between building the product and running it begins to dissolve. What the system learns from operation feeds directly back into how it operates next time.

This is an aspirational in 2026. The architectural pieces exist in isolated research contexts. The governance infrastructure for safe, auditable, autonomous self-improvement at production scale does not yet exist. We include it because the trajectory from Level 4 is visible, and teams building toward Level 4 should design with Level 5 in mind — even if they never reach it.

## 5. The Organizational Scaling Problem

---

The framework, as described, works cleanly at the individual and small-team level. Most of the evidence behind it comes from that context. Scaling to multi-team, enterprise environments introduces a distinct set of problems that the framework does not automatically solve.

### How Rules Break Down at Scale

At EP Level 4, a single practitioner accumulates a rule set that compounds across their projects. When you have five squads building across ten repositories, that model breaks. Rules generated by one squad may be unknown to another. The same error gets made, attributed, and turned into a rule five times independently.

The compounding happens, but only locally.

The fix is not complicated but requires a deliberate decision: centralized vs. federated rule of governance.

A centralized model maintains a shared rule repository that all squads inherit from, with a small platform team responsible for rule quality, conflict resolution, and propagation. A federated model allows squads to maintain domain-specific rule sets while inheriting a common baseline. In practice, most organizations need both: a core set of cross-cutting rules (security, architecture boundaries, spec quality) that are organization-wide, and domain-specific extensions that live closer to the teams that generate them.

### Platform Teams and the Harness as Infrastructure

The most effective pattern we have observed at scale is treating the harness infrastructure as a platform product.

A small platform team owns the rule baseline, governance schemas, hook architecture, and the onboarding brief generator. Product teams consume this infrastructure and contribute domain-specific rules back to it. This mirrors how platform engineering works in traditional software delivery; the analogy is not accidental.

The benefit is that EP Level 5 transferability — the property that a new practitioner can operate at Level 4 from day one — becomes an organizational capability rather than an individual one. New team members inherit not only the local project’s rule set but also the accumulated institutional knowledge of the entire organization. This is the only way to make EP 5’s governance survival test meaningful at scale: it cannot depend on one person knowing where everything lives.

## Metrics for Organizational Compounding

Individual compounding is measured by first-attempt accuracy and correction time per feature. At the organizational scale, four additional metrics matter:

- **Rule reuse rate:** what percentage of drift findings in a new project are caught by existing rules rather than generating new ADRs? A rising rate signals genuine cross-team compounding.
- **Onboarding time to EP 4 productivity:** how long does it take for a new engineer to reach Level 4 output quality? If the governance infrastructure is working, this should be days, not months.
- **Cross-squad rule propagation lag:** how long between a rule being created in one squad and its availability to others? This measures how well the governance infrastructure distributes institutional knowledge.
- **Governance debt ratio:** the ratio of postcondition failures to rule out violations in audit findings. A rising ratio signals that new failures are outpacing rule capture — the system is not compounding fast enough.

## Rule Governance at Scale

The rule lifecycle described in Section 3 — explores produces insights, refine produces provisional rules, executes persistent rules, and addresses the front end of rule management. At an organizational scale, the back end matters equally: how rules are audited, how contradictions are resolved, how abstractions replace clusters of specific rules, and how stale constraints are retired.

This is the governance-of-governance problem that expert systems and policy engines have encountered for decades. The solutions are known; the challenge is operationalizing them within the agent-assisted development workflow.

Four specific failure modes require explicit management as rule sets grow.

**Rule contradiction:** two rules that were individually correct when written conflict when applied to the same situation.

A rule blocking modification of the auth module and a rule requiring immediate application of a security patch will deadlock when a patch touches the auth files. Without a resolution mechanism, the governance system produces irreconcilable blocks. **Rule brittleness:** rules written for a monolith become actively harmful during a migration to microservices. The “never delete, only supersede” lifecycle is correct in principle, but requires a trigger for supersession, which is currently unspecified.

Rule of opacity: as the rule set grows beyond what any individual can hold in mind, violations arrive without a legible rationale, and practitioners learn to treat them as noise.

Rule redundancy: multiple rules addressing the same underlying concern accumulate into slight variations, leading to inconsistent enforcement and eroding trust in the system.

The agent-as-governor pattern addresses these failure modes by applying LLM reasoning to the semantic tasks that deterministic rule engines cannot handle. Before a new rule is committed, the agent checks against the active set for logical conflict and blocks the write if tension is detected, requiring human resolution before proceeding.

At retro time, the agent inspects rule clusters for abstraction opportunities: ten rules expressing variations of the same principle are combined into a single general rule, with the ten as examples, reducing noise without losing institutional knowledge.

Activation telemetry — tracking last-activated, activation-count, and projects-activated-in for each rule — feeds a staleness analysis that has not fired within a configurable window, marks candidates for review. The agent proposes, the human decides, and the ADR records the outcome.

Authority structure matters as much as the mechanisms.

At an organizational scale, the questions of who can add rules, who can supersede them, who can trigger abstraction, and how conflicts are escalated require explicit governance. The platform team pattern described earlier in this section is the right organizational container: the platform team owns rule quality for the shared baseline, product teams have authority over domain-specific rules, and promotion from domain-specific to shared baseline requires platform team review. Without this structure, rule governance devolves to the individual who last touched the rule file — which is no governance at all.

In Arpinine-Harness, these four metrics surface directly in /spec-status — the governance table that combines EP-level indicators (rule density, attribution counts, ADR coverage) with AIN-level indicators (eval plan status, feedback channel definition) in a single view. For teams not using Arpinine-Harness, the same metrics can be tracked manually in a shared retro log alongside each project's audit findings.

*The hardest part of scaling EP maturity is cultural, not technical. Engineers who have never operated in a compounding rule system will experience the early investment in rule-writing as overhead with no visible return. The return becomes visible around the third project. Getting teams through the first two is primarily a change management problem.*

A practical approach: start with a single high-pain workflow where the cost of recurring errors is visible and quantifiable. Run it under EP 4 governance for six weeks. Measure first-attempt accuracy and correction time before and after.

Use the numbers to support the rest of the organization's argument.

## 6. Indicators by Axis

Use these as honest diagnostic tools, not aspirational checklists. The early signal column describes what the level looks and feels like when you first get there. The mature signal column describes it at full operation. Most teams sit somewhere between the two. The diagnostic questions in the final column are the most actionable part — use them as quick team discussion prompts in a retro or planning session.

### AI Engineering Process Maturity — Indicators

| Dimension              | Early signal                                 | Mature signal                                                                   | Diagnostic question                                                                 |
|------------------------|----------------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| EP 1 — Ad-Hoc          | Occasionally uses AI for code snippets       | No AI involvement in architecture decisions; all errors caught in review        | <b>Do the same errors appear in consecutive sessions?</b>                           |
| EP 2 — Contextual      | Provides file contents before prompting      | Pre-session context ritual; quality collapses when context is thin              | <b>Can you point to a lesson from last month that changed your prompting today?</b> |
| EP 3 — Formal Planning | Writes ‘Done when...’ criteria before coding | All tasks have verifiable gates; violations are caught systematically           | <b>Is every AI ‘done’ claim verified against a stated postcondition?</b>            |
| EP 4 — Systematic      | Errors generate rule candidates              | Rule set compounds across projects; human correction time decreasing measurably | <b>Does your rule set prevent errors across projects, or only within them?</b>      |
| EP 5 — Operating Model | Governance is tracked as a managed system    | A new practitioner reaches EP 4 on day 1 from written artifacts alone           | <b>Would your governance system survive your departure?</b>                         |

## Product AI-Nativeness — Indicators

| Dimension                                | Early signal                               | Mature signal                                                       | Diagnostic question                                                                 |
|------------------------------------------|--------------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>AIN 1 — Absent</b>                    | No AI in product                           | Deliberately AI-free; no provision for AI agency                    | <b>Is AI excluded by design or by omission?</b>                                     |
| <b>AIN 2 — Feature Layer</b>             | Chatbot or recommendation engine added     | AI visible to users; core architecture designed for human operators | <b>Can an AI agent initiate operations, or only respond to user triggers?</b>       |
| <b>AIN 3 — AI-Callable</b>               | API or tool schema published               | Operations composable by agents; schemas documented                 | <b>Can an agent observe what happened after it took action?</b>                     |
| <b>AIN 4 — Composable + Feedback</b>     | Event stream or feedback channel available | Agents compose, execute, observe, and refine in a real-time loop    | <b>Does the product tell the AI what happened, or does the AI have to infer it?</b> |
| <b>AIN 5 — Self-Improving (Frontier)</b> | Agent experience feeds model improvement   | Build/operate distinction dissolving                                | <b>Does your product learn from AI-generated operations?</b>                        |

## 7. Transition Mechanics

Every transition between levels has a specific investment it requires, a characteristic way it fails, and a concrete signal that tells you the transition has happened rather than just been attempted.

| Transition | EP: from                            | EP: to                                    | AIN: from                            | AIN: to                             |
|------------|-------------------------------------|-------------------------------------------|--------------------------------------|-------------------------------------|
| EP 1 → 2   | Opportunistic prompting             | Context-rich prompting as ritual          | No structure                         | Rich pre-session context            |
| EP 2 → 3   | Quality varies by context           | Verifiable postconditions on all tasks    | Quality varies by context            | Quality verifiable against criteria |
| EP 3 → 4   | Corrections fix the session         | Corrections generate persistent rules     | Knowledge in the practitioner's head | Knowledge of machine-readable rules |
| EP 4 → 5   | Governance works for the originator | Governance transfers to new practitioners | Individual excellence                | Systemic capability                 |

| Transition | EP: from                            | EP: to                               | AIN: from                 | AIN: to                      |
|------------|-------------------------------------|--------------------------------------|---------------------------|------------------------------|
| AIN 1 → 2  | No AI in product                    | AI features visible to users         | Architecture ignores AI   | AI exists as a feature layer |
| AIN 2 → 3  | AI features; no agent interface     | AI-callable API or tool schema       | AI as a feature consumer  | AI as an operation initiator |
| AIN 3 → 4  | Agents can act; they cannot observe | Real-time feedback channel to agents | One-way command interface | Closed-loop agent operation  |
| AIN 4 → 5  | Feedback informs agent behavior     | Feedback trains agent models         | Operational feedback loop | Self-improving feedback loop |

### The Most Consequential Transitions

Two transitions are skipped more often than any others, and both are disproportionately valuable.

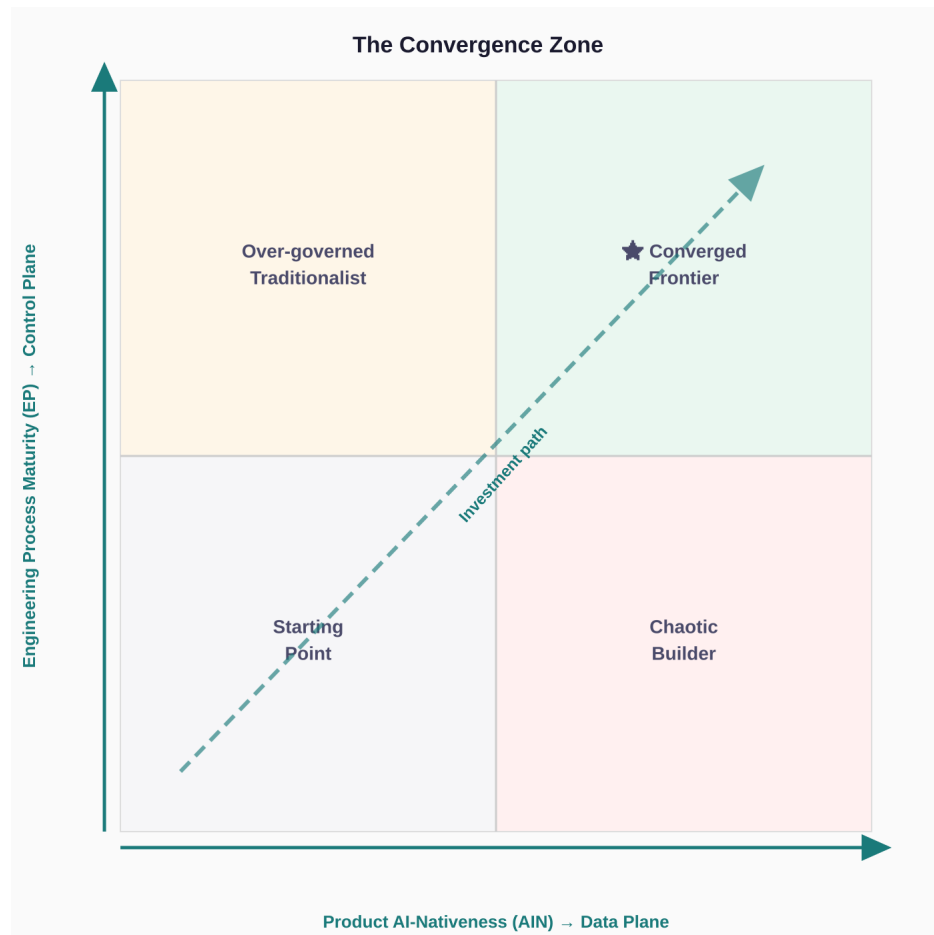
EP 3 → 4 (The compounding threshold). The gap between 3 and 4 is not capability; the AI already handles 4.

It is about whether corrections are becoming rules. Open your rule files. If they are empty or sparse, you are 3, regardless of how good your sessions feel. In Arpinine-Harness terms: if /spec-retro has never been run or specify/rules/ is empty, nothing is compounding.

AIN 2 → 3 (The architecture decision point). You cannot feature-add your way to Level 3. It requires a deliberate design decision about which vocabulary of operations an AI agent should be able to compose within your system — and that question must be answered before implementation, not discovered during it.

Arpinine-Harness enforces this through the agent\_callable\_operations section of the AIN assessment block, which must be fully populated before AIN ≥ 3 specs are unlocked.

## 8. The Convergence Zone



When EP reaches Level 5, and AIN reaches Level 4, something structurally interesting happens.

The governance infrastructure you built to direct an AI coding agent — the persistent rules, the formal contracts, the attribution discipline — becomes directly applicable to the AI agents operating inside the product you built. The same governance vocabulary that constrains how the AI writes code also constrains how the AI runs workflows.

The engineering process and the product architecture share a substrate.

*At convergence, the line between the team building the system and the system itself blurs. The product carries the same governance structures that governed its construction. That is not a metaphor — it is an architectural property you can design or miss entirely.*

Designing convergence means making a deliberate choice early: use rule formats, feedback schemas, and contract vocabulary that align with the product's own agent interfaces.

If you do, EP maturity investment compounds directly into AIN maturity at very low additional cost. If you don't, the two investments remain separate, and convergence must be engineered explicitly, which is expensive.

A concrete example of shared vocabulary in practice: an Arpinine-Harness rule file has a structured YAML header with rule-id, category, triggers, severity, and source-adr. An AIN Level 4 product that exposes its own policy layer can expose the same schema as an agent-callable operation — an agent operating inside the platform can query the active rule set, check whether a proposed action would trigger a rule violation, and adjust its workflow before executing. The rule format designed to govern human-AI collaboration during construction becomes a live policy interface for agents operating in production. Same schema, same vocabulary, two uses. That is what convergence means in practice, not as a concept.

Harness Engineering frames this as the Control Plane / Data Plane distinction. The Control Plane is policy enforcement, task scheduling, governance contracts — what EP maturity builds.

The Data Plane is an agent for runtime, state storage, sandboxed execution, and feedback channels — what AIN maturity builds. Convergence is the moment when these two planes share the same vocabulary and contracts. Investment in one starts to directly fund the other. That is the whole point.

► **Arpinine-Harness implementation** (*Arpinine — agent-align v1.4.0*)

**Convergence by design:** Arpinine-Harness’s rule format (`.specify/rules/<category>/<rule-id>.md` with structured YAML frontmatter) and ADR schema (machine-readable governs: and covers: linkage) are designed to be legible not just to the human practitioners who maintain them, but to AI agents that may eventually operate within the governed system. A platform that exposes its own governance artifacts as agent-callable operations — letting an AI agent query the active rule set, check ADR coverage, or retrieve eval thresholds before taking an action — is a system that has achieved convergence.

**The /spec-status governance table as convergence signal:** When a team can look at /spec-status and see both EP-level indicators (rule density, attribution counts, onboarding readiness) and AIN-level indicators (eval plan status, feedback channel definition, agent-callable operations) in a single view, and when those two sets of indicators are trending together rather than diverging, the system is approaching convergence. When they diverge — high AIN ambition, low EP discipline — the governance table makes the misalignment visible before it becomes expensive.

## 9. Self-Assessment Checklist

Answer honestly. The value of this checklist is entirely dependent on not gaming it. Your position is the highest level at which every question in the group receives a genuine yes.

### AI Engineering Process Maturity

- Do you provide a rich architectural context (schemas, constraints, existing code) before each AI task? (EP 2+)
- Do you write explicit ‘Done when...’ criteria before the AI begins implementation? (EP 3+)

- Is every AI ‘done’ claim verified against the stated postcondition? (EP 3+)
- When the AI makes an error, do you attribute it to a precondition or postcondition failure — and generate a persistent rule accordingly? (EP 4+)
- Is each new project measurably cheaper and more accurate than last, thanks to compounded rules? (EP 4+)
- Could a new practitioner inherit your governance system and operate at EP 4 from day one? (EP 5+)
- Do you track building economics at the process level, not just per-tool license costs? (EP 5+)

## Product AI-Nativeness

- Does your product expose a structured vocabulary that AI agents can call programmatically? (AIN 3+)
- Are the operations composable — can an agent construct multi-step workflows from them? (AIN 3+)
- Does your product provide a feedback channel that tells an agent what happened after it acted? (AIN 4+)
- Can an agent close the loop — compose, execute, observe, refine — without human involvement in each cycle? (AIN 4+)
- Does agent-generated operational data feed back into the model or agent for improvement? (AIN 5 — frontier)
- Does your agent runtime actively manage context — capping file reads, offloading oversized tool results, compacting on token pressure, and isolating sub-agent sessions? A system that achieves AIN 4 in short sessions but degrades after 30 minutes of operation has achieved a demo, not production. (AIN 4 — context continuity)

## 10. Investment Implications

Most AI company investments are going to the wrong place. Better models, more tool licenses, and prompt engineering workshops — these optimize EP 1 and 2.

They make individual sessions marginally better. They produce no compounding return because nothing in the system accumulates.

The investments that actually compound are not on most procurement dashboards.

Time spent building persistent rule sets. Formal planning infrastructure.

The discipline of attribution. Ensuring governance can be transferred to a new person.

These take a few weeks to accumulate, and then they run indefinitely. They do not require a vendor relationship.

On the product side, the AIN 2 to 3 transition is one of the few architectural decisions that is significantly cheaper before the product is built than after.

Organizations going through platform renovation have a window. Once the architecture is established and teams are built on top of it, retrofitting an AI-callable design is disruptive and expensive. The window is real, and it closes.

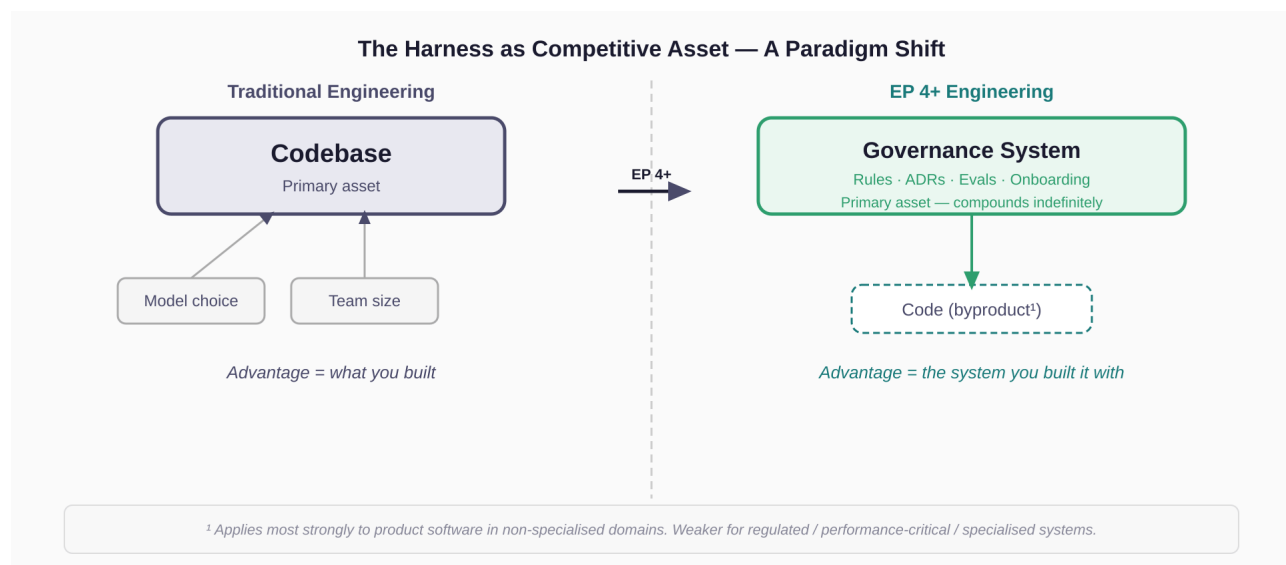
*The right question is not which model to run. It is what governance infrastructure needs to be built, and which architectural decisions need to be made now — before the window for cheap choices closes.*

## 11. The Harness as Competitive Asset

The investment implications in the previous section describe where to spend money and time.

This section addresses the deeper strategic question: what is being built when a team invests in governance infrastructure?

The answer reframes from the economics of AI-assisted engineering in a way that most organizations have not yet fully processed.



*In AI-assisted engineering, the harness is the product. The code is the output. This is not a metaphor — it is a description of where value accumulates, what is durable across model upgrades, and what cannot be copied by a competitor who simply buys the same model subscription.*

The traditional source of competitive advantage in software engineering was the codebase: the system's accumulated intellectual property, expressed as implemented functionality. That asset was hard to

replicate because it represented years of engineering effort. In AI-assisted engineering at EP Level 4 and above, the codebase is increasingly a byproduct of a faster process. A team with a mature governance system can produce in days what took months. The code itself is less durable as a competitive moat — because any team with a mature governance system can reproduce it faster, and because AI can regenerate functional equivalents from the spec and rules. This is true for most product software in non-specialized domains. It is less true in contexts where the code is itself the irreducible output — regulated systems where the specific implementation is the auditable artifact, performance-critical systems where micro-optimizations are inseparable from the value, or specialized domains where the codebase encodes years of expert judgment that cannot be specified away. The framework's investment argument applies most strongly outside those contexts.

What cannot be quickly replicated is the governance system itself: the accumulated rule set that prevents known failures; the architectural decisions with rationale that prevent re-litigation of settled questions; the attribution discipline that turns every error into a permanent constraint; the evaluation thresholds that define what “done” means across every project; and the onboarding infrastructure that transfers all of this to new practitioners on day one. This system took weeks to build and compounds indefinitely. It is not in the codebase.

It is in harness.

This reframes several decisions that organizations currently treat as peripheral. Documentation is not overhead — it is the persistence layer of the harness.

Retros are not team rituals — they are the primary mechanism by which the harness improves.

ADRs are not bureaucratic artifacts — they are the institutional memory that prevents the organization from relitigating expensive decisions.

When viewed through this lens, the investments that feel like processing overhead are investments in the primary asset.

The implication for how organizations evaluates AI engineering productivity is significant.

Velocity metrics, features per sprint, commits per week, and time-to-merge measure the system's output. They do not measure the system itself.

A team that is producing fast output while its harness degrades is liquidating an asset, not building one.

The metrics that matter are those that measure harness health: first-attempt accuracy trend, rule activation frequency, drift findings resolved via existing rules, and onboarding time to EP 4 productivity.

These are the leading indicators of whether the primary asset is appreciating or depreciating.

The same logic applies to the product's side.

A product at AIN Level 4 — with composable agent interfaces, structured feedback channels, and governed evaluation contracts — is architecturally more valuable than a product at AIN Level 2 with the same surface-level AI features.

Not because of what it does today, but because of what it enables tomorrow: agents that can operate within it, observe consequences, and refine their behavior without human mediation at every step. That architectural property is durable in a way that any specific feature is not.

Features get copied. Architecture gets compounded.

*The paradigm shift is this: in an AI-assisted engineering world, the question “what did we build?” is less important than “how did we build it, and does the system we used to build it leave us stronger than before?” A team that ships a feature and leaves the harness unchanged has delivered output. A team that ships the same feature and leaves behind three new rules, one architectural decision with rationale, and a tighter eval threshold has delivered output and invested in the asset. Over time, only one of these teams' compounds. The boundary condition: in domains where the code itself is the regulated, audited, or performance-critical artefact, both questions matter equally — what was built and how the system that built it has improved. The paradigm shift applies to the broad middle of software engineering, not to every corner of it.*

## 12. Measuring Your Position

---

Self-assessment checklists are a starting point. The more durable approach is to instrument your process so that your position on both axes becomes visible from data rather than self-report. A handful of lightweight metrics can be tracked with minimal overhead from day one.

### EP Metrics

- First-attempt acceptance rate: what fraction of AI-generated outputs are accepted without correction in the same session? Track this per project over time. A rising rate without model changes signals EP compounding.
- Human correction time per feature: clock the time from the AI output to the merged code, excluding waiting time. If this is falling across projects, the rule set is working.
- Rule activation frequency: how often does the governance system block or warn on a write? A high rate early, followed by a declining rate, suggests the system is learning. A persistently high rate suggests rules are not preventing recurrence.
- Drift findings resolved via existing rules: in each /spec-audit run, what fraction of findings match active rules vs. generate new ADRs? A rising fraction signals compounding.
- 

In Arpinine-Harness, the EP metrics above surface in /spec-status alongside AIN indicators — giving a single view of both axes trending together or diverging. For teams tracking manually, a shared retro

log with these four data points per project is sufficient to see the compounding curve within two or three projects.

## AIN Metrics

- Agent operation coverage: what percentage of your product’s core operations are exposed in the AI-callable vocabulary? A rising number signals AIN progression.
- Feedback loop latency: how long between an agent action and the product returning a structured observation? Sub-second latency enables tight iteration loops; multi-second latency makes autonomous refinement impractical.
- Loop termination rate: what fraction of agent runs terminate via a hard limit (step budget, token budget) rather than a natural completion? A high termination rate indicates loop-safety problems.

## Running Internal Experiments

The most credible way to make the case for governance investment is to run a small experiment rather than cite this paper.

Take two similar tasks. Run one under EP 2 conditions: rich context provided per session, no persistent rules, no formal completion criteria. Run the other under EP 4 conditions: write a spec with acceptance criteria first, generate a plan with module boundaries and testability gates, run /spec-audit at the end, attribute every finding, run /spec-retro, and extract at least one rule. Measure first-attempt acceptance rate and correction time for both. The difference is usually visible within a single project cycle, which is a faster feedback loop than most change management arguments get.

## Conclusion

---

The field has a vocabulary problem. Engineers who describe their AI development practices as ‘advanced’ or ‘mature’ use terms that mean different things depending on who says them.

This framework is an attempt to add precision to the conversation — not to replace judgment, but to make it easier to compare notes.

The central argument is simple: the model is not a constraint.

Two teams with identical models and access will produce radically different outcomes if one has built governance infrastructure, and the other hasn’t.

Rules that compound across sessions, contracts that make AI output falsifiable, and policy gates that enforce discipline structurally rather than relying on review — these are the investments that change the economics.

Arpinine-Harness, built by Arpinine, is the most complete working implementation of these ideas we have seen. It is not the only possible implementation — but it demonstrates that the governance structures described here are not theoretical.

They run in production, block writes, compound rules, and generate onboarding briefs from machine-readable artifacts. If you want to see what EP 4 looks like in practice, that is a reasonable place to start.

Both the framework and implementation will need refinement.

The evidence base is narrow, some of the AIN levels are aspirational, and the convergence thesis has not been demonstrated at an organizational scale.

We are publishing this now because the ideas are useful now and because the only way to refine them is to put them in front of people who will test them.

*Try self-assessment. Run one retro with rule extraction. Pick your highest-pain workflow and govern it under EP 4 conditions for six weeks. Then share your EP and AIN coordinates — and whether the transition mechanics described here matched what you actually experienced.*

## Further Reading

---

The ideas in this paper draw on and relate to a wider body of work. The following are worth reading alongside this framework.

- Harness Engineering (Hashimoto, 2026) — the foundational framing of the Agent = Model + Harness equation and the REPL Container abstraction. The Definitive Guide to Harness Engineering is the most complete treatment of the production agent infrastructure that this paper assumes.
- AI Harness Engineering Interview Preparation Handbook (AI Engineering Insider, 2026) — a practitioner-level treatment of the seven harness layers, loop architectures, sandboxing levels, and evaluation patterns. Useful for teams building the Data Plane infrastructure required by AIN Level 4.

- The original seven-tier AI-Human Engineering Maturity Model (Illsley, 2026) — the source framework this paper extends. The single-axis model and the evidence base for the 67x productivity figure are both documented there.
- Capability Maturity Model Integration (CMMI Institute) — the prior art for process maturity frameworks in software engineering. Understanding why CMMI adoption failed at scale in the 1990s is directly relevant to the organizational scaling challenges described in Section 5.
- Design by Contract (Meyer, 1992) — the formal methods foundation for the precondition/postcondition/invariant language used throughout this paper.
- Context Management in Agent Harnesses (Arize AI, 2026) — a comparative analysis of how Claude Code, Pi, OpenClaw, and Letta manage file reads, tool result offloading, session compaction, and sub-agent isolation. The convergent patterns across all four systems ground the context of the continuity requirement described in Section 4 of this paper. Directly relevant to teams designing AIN Level 4 runtime infrastructure. The convergent-evolution framing — that identical mechanisms emerge independently from the same physical constraint — is developed further in practitioner commentary by YuLoop (2026), which also introduces the proactive offloading insight and the formulation “context is not inherited, context must be designed.”

*On tooling: the harness infrastructure described in this paper can be built with custom hooks, open-source agent frameworks, or extensions to existing tools — Claude Code, Cursor, LangGraph, and similar platforms all support the hook-and-rule patterns at varying levels of maturity. Arpinine-Harness is one concrete realization; it is not the only path. What matters is that the governance structures exist and compound, not which tool implements them.*

Arpinine-Harness is available at: <https://arpinine.com/arpinine-harness>

© 2026. This white paper may be reproduced for non-commercial use with attribution.

Changelog: v1.0 (April 2026) — incorporates organizational scaling, metrics instrumentation, compliance notes, and tooling ecosystem guidance from practitioner feedback on prior drafts.